

MCTS Financiero

Estructura detallada de `mcts_tsla.py`

Guía de implementación de un agente **Monte Carlo Tree Search** para la **evaluación de posición en un activo financiero** (TSLA).

Basada en `mcts_trading.py` y `mcts_portfolio.py`,
con mejoras en rollout, métricas de riesgo y visualizaciones.

Activo de referencia: Tesla, Inc. (TSLA)

Marco algorítmico: Monte Carlo Tree Search (UCT/UCB1)

Rollout estocástico: GBM multivariado con calibración rolling

Salidas generadas: 5 figuras + tabla de métricas + reporte .md

Índice

1	Visión general del archivo	2
2	Sección 0 — Cabecera y constantes globales	2
3	Sección 1 — Espacio de acciones (<code>class Action</code>)	4
4	Sección 2 — Estado del entorno (<code>class State</code>)	4
5	Sección 3 — Nodo del árbol (<code>class Node</code>)	6
6	Sección 4 — Motor MCTS (<code>class MCTS_Agent</code>)	8
7	Sección 5 — Descarga de datos (<code>download_data</code>)	12
8	Sección 6 — Backtester (<code>class Backtester</code>)	13
9	Sección 7 — Visualizaciones (<code>class Visualizer</code>)	17
9.1	Figura 1 — Precio, señales MCTS y ratio de posición	17
9.2	Figura 2 — Evolución del portafolio y drawdown	20
9.3	Figura 3 — Distribución de retornos y rolling Sharpe	21
9.4	Figura 4 — Árbol MCTS y convergencia UCB	23
9.5	Figura 5 — Trayectorias GBM del rollout	26
10	Sección 8 — Tabla de métricas en consola	27
11	Sección 9 y 10 — Reporte y bloque principal	28
12	Resumen comparativo de mejoras	31

1. Visión general del archivo

El archivo `mcts_tsla.py` se organiza en **10 secciones numeradas**, siguiendo la misma convención de los archivos existentes en la carpeta `mcts_trading/`. La secuencia lógica es:

#	Sección	Contenido
0	Cabecera y constantes	Imports, parámetros globales configurables
1	<code>class Action</code>	Espacio de acciones discretas (5 niveles)
2	<code>class State</code>	Estado del entorno + <code>position_ratio()</code>
3	<code>class Node</code>	Nodo UCB1 + registro de historial de visitas
4	<code>class MCTS_Agent</code>	Motor MCTS: selección, expansión, rollout GBM, backprop
5	<code>download_data()</code>	Descarga via <code>yfinance</code>
6	<code>class Backtester</code>	Backtest walk-forward + métricas extendidas
7	<code>class Visualizer</code>	5 figuras con múltiples paneles
8	<code>print_metrics_table()</code>	Resumen tabular en consola
9	<code>save_report()</code>	Reporte <code>.md</code> con figuras
10	<code>__main__</code>	Bloque de entrada principal

Idea clave

Cada sección es **autocontenida**: se puede leer y modificar sin afectar a las demás. Los datos fluyen desde la sección 5 hacia abajo a través del diccionario `results` que devuelve el `Backtester`.

2. Sección 0 — Cabecera y constantes globales

Todos los parámetros que el usuario querrá ajustar se concentran aquí, en constantes con nombre, para no tener *magic numbers* dispersos en el código.

```

1  """
2  mcts_tsla.py
3  =====
4  Evaluacion de posicion en TSLA mediante Monte Carlo Tree
   Search.
5
6  Estructura:
7      0. Cabecera y constantes globales
8      1. Action          -- espacio de acciones discretas

```

```

9      2. State          -- estado del entorno financiero
10     3. Node          -- nodo del arbol con UCB1
11     4. MCTS_Agent    -- motor MCTS (4 pasos clasicos + GBM
        rolling)
12     5. download_data -- descarga de datos con yfinance
13     6. Backtester    -- backtest walk-forward y metricas
14     7. Visualizer    -- 5 figuras representativas
15     8. print_metrics_table -- resumen en consola
16     9. save_report   -- reporte Markdown
17    10. __main__      -- punto de entrada
18    """
19    from __future__ import annotations
20
21    import math
22    import os
23    import datetime
24    from dataclasses import dataclass, field
25    from enum import Enum, auto
26    from typing import Optional
27
28    import numpy as np
29    import pandas as pd
30    import matplotlib.pyplot as plt
31    import matplotlib.gridspec as gridspec
32    from matplotlib.lines import Line2D
33    from scipy import stats          # para Q-Q plot y VaR
        parametrico
34    import yfinance as yf
35
36    # -- Parametros globales (modificar aqui)
        -----
37    C_UCB:          float = math.sqrt(2)    # constante
        exploracion UCB1 (estandar)
38    RISK_FREE_ANNUAL: float = 0.045        # tasa libre de
        riesgo anual (~T-bill)
39    TICKER:         str    = "TSLA"
40    PERIOD:         str    = "2y"          # periodo de
        descarga yfinance
41    INITIAL_CASH:   float = 10_000.0        # capital inicial en
        USD
42    RESULTS_DIR:    str    = "results_tsla" # directorio de
        salida

```

Listing 1: Cabecera e imports de mcts_tsla.py

Nota

`scipy` se usa únicamente en la sección 7 para el Q-Q plot y el VaR paramétrico. Si no está instalado, se puede eliminar ese panel concreto sin afectar al resto del código.

3. Sección 1 — Espacio de acciones (class Action)

Se amplía de 3 a **5 acciones** respecto a `mcts_trading.py`. Más granularidad permite al árbol expresar convicciones de distinta intensidad.

```

1 class Action(Enum):
2     """Acciones disponibles en cada paso de decision del
3     agente."""
4     BUY_25 = auto() # invertir 25% del efectivo disponible
5     BUY_10 = auto() # invertir 10% del efectivo disponible
6     HOLD   = auto() # mantener posicion sin operar
7     SELL_10 = auto() # vender 10% de las acciones en
8     cartera
9     SELL_25 = auto() # vender 25% de las acciones en
10    cartera
11
12 ACTIONS_ALL: list[Action] = list(Action)

```

Listing 2: Definición del espacio de acciones

Idea clave

El árbol MCTS tiene ahora **5 ramas posibles** en cada nodo. Con `iterations=300`, cada acción recibe de media ~ 60 visitas antes de que se tome la decisión, suficiente para que UCB1 discrimine con fiabilidad entre comprar fuerte, comprar ligero, mantener o vender.

4. Sección 2 — Estado del entorno (class State)

El estado es *immutable* (`frozen=True`) para que los nodos del árbol nunca compartan referencias accidentalmente.

```

1 @dataclass(frozen=True)
2 class State:
3     """
4     Estado completo del entorno financiero en un instante
5     dado.
6
7     Attributes
8     -----
9     day      : Indice del dia actual en la serie de precios.
10    cash     : Efectivo disponible (USD).
11    shares  : Acciones en propiedad (puede ser fraccionario).
12    price   : Precio de cierre del activo en este dia.
13    """
14    day:      int
15    cash:     float
16    shares:  float

```

```

16     price:    float
17
18     def portfolio_value(self) -> float:
19         """Valor total: efectivo + valor de mercado de las acciones."""
20         return self.cash + self.shares * self.price
21
22     def position_ratio(self) -> float:
23         """
24         Fraccion del portfolio invertida en el activo (0.0 -
25         1.0).
26         Es la metrica central de 'evaluacion de posicion'.
27         """
28         invested = self.shares * self.price
29         total    = self.portfolio_value()
30         return invested / (total + 1e-10)
31
32     def apply_action(
33         self,
34         action:      Action,
35         next_price:  float,
36         tx_cost:     float = 0.001,    # 0.1% del capital
37         operado
38     ) -> "State":
39         """
40         Aplica una accion, cobra coste de transaccion y
41         avanza un dia.
42
43         Orden de eventos
44         -----
45         1. Calcular capital operado segun la accion.
46         2. Descontar coste de transaccion proporcional.
47         3. Actualizar cash y shares.
48         4. Devolver nuevo State con next_price y day+1.
49
50         Parameters
51         -----
52         action      : Accion a ejecutar.
53         next_price  : Precio del activo al dia siguiente.
54         tx_cost     : Fraccion del capital operado cobrada
55                         como comision.
56         """
57         cash, shares = self.cash, self.shares
58
59         if action == Action.BUY_25:
60             amount    = cash * 0.25
61             cost       = amount * tx_cost
62             shares    += (amount - cost) / self.price if self.
63                 price > 0 else 0.0
64             cash      -= amount

```

```

61         elif action == Action.BUY_10:
62             amount = cash * 0.10
63             cost = amount * tx_cost
64             shares += (amount - cost) / self.price if self.
                price > 0 else 0.0
65             cash -= amount
66
67         elif action == Action.SELL_10:
68             sold = shares * 0.10
69             proceeds = sold * self.price
70             cost = proceeds * tx_cost
71             shares -= sold
72             cash += proceeds - cost
73
74         elif action == Action.SELL_25:
75             sold = shares * 0.25
76             proceeds = sold * self.price
77             cost = proceeds * tx_cost
78             shares -= sold
79             cash += proceeds - cost
80
81         # HOLD: sin cambios
82
83         return State(
84             day=self.day + 1,
85             cash=max(cash, 0.0),
86             shares=max(shares, 0.0),
87             price=next_price,
88         )

```

Listing 3: Clase State con coste de transacción y ratio de posición

5. Sección 3 — Nodo del árbol (class Node)

Idéntico en estructura a `mcts_trading.py`, con un atributo adicional `ucb_trace` que registra el valor UCB1 en cada visita para poder graficar la convergencia del árbol (Figura 4).

```

1  class Node:
2      """
3      Nodo del arbol de busqueda MCTS.
4
5      Attributes
6      -----
7      state          : Estado del entorno en este nodo.
8      action         : Accion que llevo al padre a este nodo (None
                        en la raiz).
9      parent         : Nodo padre.
10     children        : Nodos hijo ya expandidos.

```

```

11     n           : Visitas totales a este nodo.
12     w           : Recompensa acumulada (suma de todos los
13                   rollouts).
14     ucb_trace  : Lista con el valor UCB1 registrado en cada
15                   visita.
16                   Usado para el grafico de convergencia (
17                   Figura 4).
18     """
19
20     def __init__(
21         self,
22         state: State,
23         action: Optional[Action] = None,
24         parent: Optional["Node"] = None,
25     ) -> None:
26         self.state: State = state
27         self.action: Optional[Action] = action
28         self.parent: Optional["Node"] = parent
29         self.children: list["Node"] = []
30         self.n: int = 0
31         self.w: float = 0.0
32         self.ucb_trace: list[float] = [] # NUEVO:
33             historial de UCB1
34
35     def ucb1(self, c: float = C_UCB) -> float:
36         """
37         Valor UCT del nodo.
38          $UCT_i = w_i/n_i + c * \sqrt{\ln(N_{padre}) / n_i}$ 
39         Devuelve +inf si n=0 (garantiza exploracion inicial).
40         """
41         if self.n == 0:
42             return math.inf
43         expl = self.w / self.n
44         if self.parent is None or self.parent.n == 0:
45             return expl
46         return expl + c * math.sqrt(math.log(self.parent.n) /
47                                     self.n)
48
49     def best_child(self, c: float = C_UCB) -> "Node":
50         return max(self.children, key=lambda ch: ch.ucb1(c))
51
52     def tried_actions(self) -> set[Action]:
53         return {ch.action for ch in self.children if ch.
54                 action is not None}
55
56     def untried_actions(self) -> list[Action]:
57         tried = self.tried_actions()
58         return [a for a in ACTIONS_ALL if a not in tried]
59
60     def is_fully_expanded(self) -> bool:
61         return len(self.untried_actions()) == 0

```



```

56
57     def add_child(self, action: Action, state: State) -> "
      Node":
58         child = Node(state=state, action=action, parent=self)
59         self.children.append(child)
60         return child

```

Listing 4: Clase Node con registro de convergencia UCB

6. Sección 4 — Motor MCTS (class MCTS_Agent)

Esta es la sección central. Implementa los 4 **pasos clásicos** con dos mejoras clave sobre `mcts_trading.py`:

1. **Calibración rolling**: μ y σ se estiman con los últimos `window` días, no con toda la serie.
2. **Sin lookback bias**: la expansión usa un precio GBM simulado, no el precio histórico real del día siguiente.

```

1  class MCTS_Agent:
2      """
3      Agente MCTS para decisiones de trading sobre un activo.
4
5      Mejoras sobre mcts_trading.py
6      -----
7      - Rollout GBM con calibracion rolling (mu, sigma de
        ventana movil).
8      - Recompensa = log-retorno medio (mas estable
        numericamente).
9      - Sin lookback bias en la expansion.
10     - iter_rewards: lista de recompensas por iteracion para
        grafico.
11     """
12
13     def __init__(
14         self,
15         prices: np.ndarray,
16         iterations: int = 300,
17         rollout_days: int = 20,
18         window: int = 40,
19         c_ucb: float = C_UCB,
20         tx_cost: float = 0.001,
21         rng_seed: int = 42,
22     ) -> None:
23         self.prices = prices
24         self.iterations = iterations
25         self.rollout_days = rollout_days

```

```

26     self.window          = window
27     self.c_ucb           = c_ucb
28     self.tx_cost         = tx_cost
29     self.rng             = np.random.default_rng(rng_seed)
30
31     def _calibrate(self, day: int) -> tuple[float, float]:
32         """
33         Estima mu (drift diario) y sigma (volatilidad diaria)
34         con una ventana rolling de 'self.window' dias.
35
36         Returns
37         -----
38         (mu_daily, sigma_daily) -- ambos en escala diaria.
39         """
40         start = max(0, day - self.window)
41         end   = min(day + 1, len(self.prices))
42         win   = self.prices[start:end]
43
44         if len(win) >= 3:
45             log_ret = np.diff(np.log(win))
46             mu      = float(np.mean(log_ret))
47             sigma   = float(np.std(log_ret, ddof=1)) + 1e-8
48         else:
49             mu, sigma = 0.0, 0.01
50
51         return mu, sigma

```

Listing 5: Clase MCTS_Agent – constructor y calibración

```

1     # -- Paso 1: Seleccion
2     -----
3
4     def _select(self, node: Node) -> Node:
5         """Navega el arbol con UCB1 hasta un nodo no
6            expandido."""
7         current = node
8         while current.is_fully_expanded() and current.
9             children:
10             current = current.best_child(self.c_ucb)
11         return current
12
13     # -- Paso 2: Expansion (sin lookback bias)
14     -----
15
16     def _expand(self, node: Node) -> Node:
17         """
18         Crea un hijo para una accion no probada.
19
20         Para evitar lookback bias, el precio del siguiente
21         dia se simula
22         con un paso GBM calibrado -- no se usa el precio
23         historico real.

```

```

18     """
19     untried = node.untried_actions()
20     action = self.rng.choice(untried)    # type: ignore[
        arg-type]
21
22     day      = node.state.day
23     mu, sigma = self._calibrate(day)
24     mu_ann    = mu * 252
25     sigma_ann = sigma * math.sqrt(252)
26
27     T        = 1.0 / 252.0                # 1 día en años
28     z        = self.rng.standard_normal()
29     log_ret   = (mu_ann - 0.5 * sigma_ann**2) * T +
        sigma_ann * math.sqrt(T) * z
30     next_price = float(node.state.price * math.exp(
        log_ret))
31     next_price = max(next_price, 0.01)    # precio siempre
        positivo
32
33     new_state = node.state.apply_action(action,
        next_price, self.tx_cost)
34     return node.add_child(action, new_state)

```

Listing 6: Pasos 1 y 2 – Selección y Expansión (sin lookback bias)

```

1     # -- Paso 3: Rollout (simulación GBM exacta)
        -----
2
3     def _rollout(self, node: Node) -> float:
4         """
5         Estima el log-retorno de la cartera con GBM exacto.
6
7         Modelo (solución analítica, sin error de
            discretización):
8              $S_T = S_0 * \exp((\mu - \sigma^2/2)*T + \sigma*\sqrt{T}
                )*Z)$ 
9              $Z \sim N(0,1)$ 
10
11         Se simulan  $N\_PATHS$  trayectorias en paralelo y se
            promedia,
12         lo que reduce la varianza del estimador Monte Carlo.
13
14         Recompensa = log-retorno medio de la cartera al
            horizonte.
15         (más estable numéricamente que el valor absoluto)
16         """
17         N_PATHS = 50
18         state    = node.state
19         S0       = state.price
20         day      = state.day
21
22         mu, sigma = self._calibrate(day)

```

```

23     mu_ann      = mu * 252
24     sigma_ann   = sigma * math.sqrt(252)
25     T           = self.rollout_days / 252.0
26
27     Z           = self.rng.standard_normal(N_PATHS)
28     S_T         = S0 * np.exp(
29         (mu_ann - 0.5 * sigma_ann**2) * T
30         + sigma_ann * math.sqrt(T) * Z
31     )
32
33     # Valor de la cartera al horizonte en cada
34     trayectoria
35     port_values = state.cash + state.shares * S_T
36     port_values = np.maximum(port_values, 1e-6)
37
38     # Log-retorno medio como recompensa
39     pv0          = state.portfolio_value()
40     log_returns  = np.log(port_values / (pv0 + 1e-10))
41     return float(np.mean(log_returns))

```

Listing 7: Paso 3 – Rollout GBM exacto con 50 trayectorias

```

1     # -- Paso 4: Retropropagacion
2     -----
3
4     def _backpropagate(self, node: Node, reward: float) ->
5         None:
6         """Propaga la recompensa hacia la raiz actualizando n
7         y w."""
8         current: Optional[Node] = node
9         while current is not None:
10             current.n += 1
11             current.w += reward
12             # Registrar UCB1 en el historial del nodo hijo de
13             la raiz
14             if current.parent is not None and current.parent.
15                 parent is None:
16                 current.ucb_trace.append(current.ucb1(self.
17                     c_ucb))
18             current = current.parent
19
20     # -- Bucle principal
21     -----
22
23     def run(self, root_state: State) -> tuple[Node, list[
24         float]]:
25         """
26         Ejecuta 'self.iterations' ciclos MCTS.
27
28         Returns
29         -----

```

```

22     root          : Raiz del arbol construido.
23     iter_rewards  : Recompensa de cada rollout (para
                       grafico convergencia).
24     """
25     root          = Node(state=root_state)
26     iter_rewards: list[float] = []
27
28     for _ in range(self.iterations):
29         leaf      = self._select(root)
30         if not leaf.is_fully_expanded():
31             leaf = self._expand(leaf)
32         reward = self._rollout(leaf)
33         self._backpropagate(leaf, reward)
34         iter_rewards.append(reward)
35
36     return root, iter_rewards
37
38     def best_action(self, root: Node) -> Action:
39         """Accion optima: hijo con mayor Q media (c=0,
                       explotacion pura)."""
40         if not root.children:
41             return Action.HOLD
42         best = root.best_child(c=0.0)
43         return best.action if best.action is not None else
                       Action.HOLD

```

Listing 8: Paso 4 y bucle principal del MCTS

7. Sección 5 — Descarga de datos (download_data)

```

1  def download_data(
2      ticker: str = TICKER,
3      period: str = PERIOD,
4  ) -> tuple[np.ndarray, pd.DatetimeIndex]:
5      """
6      Descarga precios de cierre ajustados.
7
8      Returns
9      -----
10     prices : np.ndarray shape (T,)    -- precios de cierre
           cronologicos.
11     dates  : pd.DatetimeIndex (T,)    -- fechas
           correspondientes.
12
13     Raises
14     -----
15     ValueError si yfinance no devuelve datos para el ticker.
16     """

```

```

17 print(f"[INFO] Descargando {ticker} ({period})...")
18 df = yf.download(ticker, period=period, progress=False,
19                 auto_adjust=True)
20
21 if df.empty:
22     raise ValueError(f"Sin datos para '{ticker}'.")
23
24 prices = df["Close"].dropna().to_numpy(dtype=float).
25         flatten()
26 dates = df["Close"].dropna().index
27
28 print(f"[INFO] {len(prices)} sesiones. "
29       f"Rango: {dates[0].date()} a {dates[-1].date()}. "
30       f"Precio actual: {prices[-1]:.2f} $")
31 return prices, dates

```

Listing 9: Descarga de precios con yfinance

8. Sección 6 — Backtester (class Backtester)

El Backtester ejecuta el agente MCTS en modo *walk-forward* (sin mirar el futuro), registrando toda la información necesaria para las visualizaciones.

```

1 class Backtester:
2     """
3     Evaluacion walk-forward del agente MCTS sobre datos
4     historicos.
5     Compara con Buy & Hold. Registra todo lo necesario para
6     Visualizer.
7     """
8
9     def __init__(
10         self,
11         mcts_iterations: int = 300,
12         rollout_days: int = 20,
13         window: int = 40,
14         c_ucb: float = C_UCB,
15         tx_cost: float = 0.001,
16         rng_seed: int = 42,
17     ) -> None:
18         self.mcts_iterations = mcts_iterations
19         self.rollout_days = rollout_days
20         self.window = window
21         self.c_ucb = c_ucb
22         self.tx_cost = tx_cost
23         self.rng_seed = rng_seed
24
25     def run(
26         self,

```

```

25     prices:          np.ndarray,
26     dates:           pd.DatetimeIndex,
27     initial_cash:    float = INITIAL_CASH,
28 ) -> dict:
29     """
30     Backtest completo. Devuelve un diccionario con todas
31     las series
32     temporales y estructuras necesarias para las
33     visualizaciones.
34
35     Campos del diccionario devuelto
36     -----
37     portfolio_series    list[float]          valor cartera
38                        MCTS cada dia
39     bah_series          list[float]          valor Buy & Hold
40                        cada dia
41     actions             list[Action]         accion tomada en
42                        cada dia
43     position_ratios     list[float]          % invertido en
44                        TSLA cada dia
45     iter_rewards_last   list[float]          recompensas por
46                        iteracion (ultimo dia)
47     ucb_root_last       Node                 raiz del arbol
48                        del ultimo dia
49     gbm_paths_last      np.ndarray (50,T)    trayectorias GBM
50                        del ultimo dia
51     dates               pd.DatetimeIndex     fechas de la
52                        serie
53     initial_cash        float
54     """
55
56     agent = MCTS_Agent(
57         prices          = prices,
58         iterations      = self.mcts_iterations,
59         rollout_days    = self.rollout_days,
60         window          = self.window,
61         c_ucb           = self.c_ucb,
62         tx_cost         = self.tx_cost,
63         rng_seed        = self.rng_seed,
64     )
65
66     state = State(day=0, cash=initial_cash, shares=0.0,
67                   price=float(prices[0]))
68
69     portfolio_series: list[float] = [initial_cash]
70     actions:          list[Action] = []
71     position_ratios:  list[float]  = [0.0]
72     iter_rewards_last: list[float]  = []
73     ucb_root_last:    Optional[Node] = None
74
75     for day in range(len(prices) - 1):
76         state = State(day=day,

```

```

66         cash=portfolio_series[-1] * (1 -
67             position_ratios[-1]),
68         shares=portfolio_series[-1] *
69             position_ratios[-1]
70             / float(prices[day]),
71         price=float(prices[day]))
72
73     root, iter_rewards = agent.run(state)
74     action = agent.best_action(root)
75     actions.append(action)
76
77     next_price = float(prices[day + 1])
78     state = state.apply_action(action,
79         next_price, self.tx_cost)
80
81     portfolio_series.append(state.portfolio_value())
82     position_ratios.append(state.position_ratio())
83
84     # Guardar el estado del ultimo dia para las
85     visualizaciones
86     if day == len(prices) - 2:
87         iter_rewards_last = iter_rewards
88         ucb_root_last = root
89         # Generar trayectorias GBM del ultimo dia
90         para Figura 5
91         gbm_paths_last = agent._simulate_paths(state,
92             prices)
93
94     pct = (day + 1) / (len(prices) - 1) * 100
95     print(f"\r[MCTS] {day+1}/{len(prices)-1} ({pct:.0f}%) "
96         f"Cartera: {state.portfolio_value():>10,.0f}$ "
97         f"Accion: {action.name:<8}", end="", flush=
98             True)
99
100     print()
101
102     # Buy & Hold
103     shares_bah = initial_cash / prices[0]
104     bah_series = [float(shares_bah * p) for p in prices]
105
106     return {
107         "portfolio_series": portfolio_series,
108         "bah_series": bah_series,
109         "actions": actions,
110         "position_ratios": position_ratios,
111         "iter_rewards_last": iter_rewards_last,
112         "ucb_root_last": ucb_root_last,
113         "gbm_paths_last": gbm_paths_last,
114         "dates": dates,
115         "initial_cash": initial_cash,

```


108 }

Listing 10: Constructor y metodo run() del Backtester

```

1  @staticmethod
2  def compute_metrics(
3      portfolio_series: list[float],
4      label: str = "",
5      risk_free: float = RISK_FREE_ANNUAL,
6  ) -> dict:
7      """
8      Calcula el conjunto completo de metricas financieras.
9
10     Metricas devueltas
11     -----
12     roi          : Retorno total acumulado (fraccion).
13     cagr         : Retorno anual compuesto (fraccion).
14     sharpe       : Ratio de Sharpe anualizado.
15     sortino      : Ratio de Sortino anualizado (solo vol.
16                   negativa).
17     max_dd       : Maximo Drawdown (fraccion negativa).
18     calmar       : CAGR / |Max Drawdown| (ratio riesgo-
19                   retorno largo plazo).
20     var_95       : Value at Risk 95% diario parametrico (
21                   fraccion).
22     cvar_95      : Expected Shortfall 95% (media de
23                   perdidas > VaR).
24     final        : Valor final de la cartera.
25     """
26     arr = np.array(portfolio_series, dtype=float)
27     ret = np.diff(arr) / arr[:-1] # retornos
28         diarios
29     rf_d = risk_free / 252 # tasa
30         libre de riesgo diaria
31
32     # Retorno y CAGR
33     roi = float((arr[-1] - arr[0]) / arr[0])
34     n_days = len(arr)
35     cagr = float((arr[-1] / arr[0]) ** (252 / n_days) -
36                1)
37
38     # Sharpe
39     mean_r = float(np.mean(ret))
40     std_r = float(np.std(ret, ddof=1)) + 1e-10
41     sharpe = (mean_r - rf_d) / std_r * math.sqrt(252)
42
43     # Sortino (solo volatilidad a la baja)
44     neg = ret[ret < rf_d]
45     down_std = float(np.std(neg, ddof=1)) + 1e-10 if len(
46         neg) > 1 else std_r
47     sortino = (mean_r - rf_d) / down_std * math.sqrt
48         (252)

```

```

40
41     # Maximo Drawdown
42     peak      = np.maximum.accumulate(arr)
43     dd        = (arr - peak) / peak
44     max_dd    = float(np.min(dd))
45
46     # Calmar
47     calmar    = cagr / abs(max_dd) if max_dd != 0 else float
48               ("inf")
49
50     # VaR y CVaR 95% parametrico (distribucion normal)
51     z95       = stats.norm.ppf(0.05)
52     var_95    = float(mean_r + z95 * std_r)
53     cvar_mask = ret <= var_95
54     cvar_95    = float(np.mean(ret[cvar_mask])) if
55                   cvar_mask.any() else var_95
56
57     return {
58         "label":    label,
59         "roi":      roi,
60         "cagr":     cagr,
61         "sharpe":   sharpe,
62         "sortino":  sortino,
63         "max_dd":   max_dd,
64         "calmar":   calmar,
65         "var_95":   var_95,
66         "cvar_95":  cvar_95,
67         "final":    float(arr[-1]),
68     }

```

Listing 11: Metodo compute_metrics() – métricas extendidas de riesgo

9. Sección 7 — Visualizaciones (class Visualizer)

Se generan 5 figuras independientes, cada una guardada como .png de 150 dpi en RESULTS_DIR.

9.1. Figura 1 — Precio, señales MCTS y ratio de posición

Fig. 1: fig1_signals.png

- **Panel superior (60 %):** Precio de TSLA con marcadores de acción superpuestos (▲ verde=BUY, ▼ rojo=SELL, ● azul=HOLD). Incluye media móvil de 20 sesiones.
- **Panel inferior (40 %):** Ratio de posición (% del portfolio invertido en TSLA) como área rellena azul. Permite ver si el agente acumula o reduce exposición a lo

largo del tiempo.

Qué muestra: la señal MCTS en contexto del precio y el nivel de exposición resultante.

```

1  class Visualizer:
2      """Genera las 5 figuras del analisis MCTS."""
3
4      ACTION_COLORS = {
5          Action.BUY_25: "#1a7a1a", # verde oscuro
6          Action.BUY_10: "#2ecc71", # verde claro
7          Action.HOLD: "#3498db", # azul
8          Action.SELL_10: "#e74c3c", # rojo claro
9          Action.SELL_25: "#8e0000", # rojo oscuro
10     }
11     ACTION_MARKERS = {
12         Action.BUY_25: "^",
13         Action.BUY_10: "^",
14         Action.HOLD: ".",
15         Action.SELL_10: "v",
16         Action.SELL_25: "v",
17     }
18
19     def __init__(self, results: dict, prices: np.ndarray,
20                 dates: pd.DatetimeIndex, ticker: str,
21                 output_dir: str = RESULTS_DIR) -> None:
22         self.results = results
23         self.prices = prices
24         self.dates = dates
25         self.ticker = ticker
26         self.output_dir = output_dir
27         os.makedirs(output_dir, exist_ok=True)
28
29     def plot_signals(self) -> None:
30         """Fig 1: precio con senales MCTS + ratio de posicion"""
31
32         actions = self.results["actions"]
33         pos_rat = self.results["position_ratios"]
34
35         fig, (ax1, ax2) = plt.subplots(
36             2, 1, figsize=(14, 8),
37             gridspec_kw={"height_ratios": [3, 2]},
38             sharex=True,
39         )
40         fig.suptitle(
41             f"MCTS Trading -- Senales sobre {self.ticker}",
42             fontsize=13, fontweight="bold",
43         )
44
45         # Panel 1: Precio + senales + MA20
46         ax1.plot(self.dates, self.prices, color="#2c3e50",

```

```

46         lw=1.4, label=f"Precio {self.ticker}",
           zorder=2)
47     ma20 = pd.Series(self.prices).rolling(20).mean().
           values
48     ax1.plot(self.dates, ma20, color="#e67e22", lw=1.0,
49             ls="--", alpha=0.8, label="MA 20d")
50
51     for i, action in enumerate(actions):
52         ax1.scatter(self.dates[i], self.prices[i],
53                   color=self.ACTION_COLORS[action],
54                   marker=self.ACTION_MARKERS[action],
55                   s=18, zorder=3, alpha=0.7)
56
57     legend_elements = [
58         Line2D([0],[0], marker="^", color="w",
59               markerfacecolor="#1a7a1a", markersize=8,
60               label="BUY 25%"),
61         Line2D([0],[0], marker="^", color="w",
62               markerfacecolor="#2ecc71", markersize=8,
63               label="BUY 10%"),
64         Line2D([0],[0], marker=".", color="w",
65               markerfacecolor="#3498db", markersize=8,
66               label="HOLD"),
67         Line2D([0],[0], marker="v", color="w",
68               markerfacecolor="#e74c3c", markersize=8,
69               label="SELL 10%"),
70         Line2D([0],[0], marker="v", color="w",
71               markerfacecolor="#8e0000", markersize=8,
72               label="SELL 25%"),
73     ]
74     ax1.legend(handles=legend_elements, fontsize=8, loc="
75               upper left")
76     ax1.set_ylabel(f"Precio {self.ticker} [$]")
77     ax1.grid(alpha=0.3)
78
79     # Panel 2: Ratio de posicion
80     ax2.fill_between(self.dates, pos_rat,
81                    color="#3498db", alpha=0.4, label="%
82                    en TSLA")
83     ax2.plot(self.dates, pos_rat, color="#2980b9", lw
84             =1.0)
85     ax2.set_ylim(0, 1.05)
86     ax2.set_ylabel("Ratio de posicion")
87     ax2.set_xlabel("Fecha")
88     ax2.legend(fontsize=9)
89     ax2.grid(alpha=0.3)
90
91     plt.tight_layout()
92     path = os.path.join(self.output_dir, "fig1_signals.
93               png")
94     plt.savefig(path, dpi=150, bbox_inches="tight")

```

```

86     plt.close()
87     print(f"[OK] {path}")

```

Listing 12: Figura 1 – senales y ratio de posicion

9.2. Figura 2 — Evolución del portafolio y drawdown

Fig. 2: fig2_performance.png

- **Panel superior (55 %):** Valor normalizado (base 100) del portafolio MCTS y Buy & Hold con línea de referencia a 100.
- **Panel inferior (45 %):** Drawdown (%) del portafolio MCTS (área roja rellena) y de B&H (línea punteada naranja).

Qué muestra: comparación directa de performance y resistencia a caídas.

```

1  def plot_performance(self) -> None:
2      """Fig 2: evolución de cartera (base 100) + drawdown.
3          """
4      pv = np.array(self.results["portfolio_series"])
5      bah = np.array(self.results["bah_series"])
6      b = pv[0]
7      pv_n = pv / b * 100
8      bah_n = bah / b * 100
9
10     fig, (ax1, ax2) = plt.subplots(
11         2, 1, figsize=(14, 8),
12         gridspec_kw={"height_ratios": [11, 9]},
13         sharex=True,
14     )
15     fig.suptitle(
16         f"Evolucion del portafolio -- MCTS vs Buy & Hold
17         ({self.ticker})",
18         fontsize=13, fontweight="bold",
19     )
20
21     # Panel 1: valor normalizado
22     ax1.plot(self.dates, pv_n, lw=2.0, color="#1f77b4",
23             label="MCTS Agent", zorder=3)
24     ax1.plot(self.dates, bah_n, lw=1.5, color="#ff7f0e",
25             ls="--", label="Buy & Hold")
26     ax1.axhline(100, color="gray", lw=0.8, ls=":")
27     ax1.set_ylabel("Valor (base 100)")
28     ax1.legend(fontsize=10)
29     ax1.grid(alpha=0.3)
30
31     # Panel 2: drawdown
32     peak_mcts = np.maximum.accumulate(pv)
33     dd_mcts = (pv - peak_mcts) / peak_mcts * 100
34     peak_bah = np.maximum.accumulate(bah)

```

```

33     dd_bah      = (bah - peak_bah) / peak_bah * 100
34     ax2.fill_between(self.dates, dd_mcts, 0,
35                      color="#d62728", alpha=0.5, label="
                        DD MCTS")
36     ax2.plot(self.dates, dd_bah, color="#ff7f0e",
37              lw=1.0, ls="--", alpha=0.8, label="DD B&H")
38     ax2.set_ylabel("Drawdown (%)")
39     ax2.set_xlabel("Fecha")
40     ax2.legend(fontsize=9)
41     ax2.grid(alpha=0.3)
42
43     plt.tight_layout()
44     path = os.path.join(self.output_dir, "
                        fig2_performance.png")
45     plt.savefig(path, dpi=150, bbox_inches="tight")
46     plt.close()
47     print(f"[OK] {path}")

```

Listing 13: Figura 2 – performance y drawdown

9.3. Figura 3 — Distribución de retornos y rolling Sharpe

Fig. 3: fig3_returns.png

- **Panel izquierdo superior:** Histograma de retornos diarios MCTS vs B&H con curva normal ajustada. Permite detectar asimetría (skewness) y colas gruesas (kurtosis).
- **Panel derecho superior:** Q-Q plot de los retornos MCTS contra la distribución normal teórica.
- **Panel inferior (fila completa):** Rolling Sharpe de 30 días para MCTS y B&H. Muestra en qué períodos el agente fue más eficiente.

```

1  def plot_returns(self) -> None:
2      """Fig 3: histograma, Q-Q plot y rolling Sharpe."""
3      pv = np.array(self.results["portfolio_series"])
4      bah = np.array(self.results["bah_series"])
5      ret_mcts = np.diff(pv) / pv[:-1]
6      ret_bah = np.diff(bah) / bah[:-1]
7
8      fig = plt.figure(figsize=(14, 9))
9      fig.suptitle(f"Análisis de retornos -- {self.ticker}"
10                  ,
11                  fontsize=13, fontweight="bold")
12      gs = gridspec.GridSpec(2, 2, hspace=0.42, wspace
13                             =0.32)
14
15      # Panel 1: histograma
16      ax1 = fig.add_subplot(gs[0, 0])
17      bins = 50

```

```

16     ax1.hist(ret_mcts, bins=bins, density=True, alpha
17             =0.6,
18             color="#1f77b4", label="MCTS")
19     ax1.hist(ret_bah, bins=bins, density=True, alpha=0.4,
20             color="#ff7f0e", label="B&H")
21     # Curva normal ajustada a MCTS
22     mu_r, s_r = float(np.mean(ret_mcts)), float(np.std(
23                 ret_mcts))
24     xs = np.linspace(ret_mcts.min(), ret_mcts.max(), 200)
25     ax1.plot(xs, stats.norm.pdf(xs, mu_r, s_r),
26             "k--", lw=1.2, label="Normal fit")
27     ax1.set_title("Distribucion de retornos diarios")
28     ax1.set_xlabel("Retorno diario")
29     ax1.legend(fontsize=8)
30     ax1.grid(alpha=0.3)
31
32     # Panel 2: Q-Q plot
33     ax2 = fig.add_subplot(gs[0, 1])
34     (osm, osr), (slope, intercept, _) = stats.probplot(
35         ret_mcts)
36     ax2.scatter(osm, osr, s=6, color="#1f77b4", alpha
37                =0.6)
38     ax2.plot(osm, slope * np.array(osm) + intercept,
39             "r--", lw=1.2, label="Recta normal")
40     ax2.set_title("Q-Q plot retornos MCTS")
41     ax2.set_xlabel("Cuantiles teoricos")
42     ax2.set_ylabel("Cuantiles observados")
43     ax2.legend(fontsize=8)
44     ax2.grid(alpha=0.3)
45
46     # Panel 3: rolling Sharpe 30d
47     ax3 = fig.add_subplot(gs[1, :])
48     rf_d = RISK_FREE_ANNUAL / 252
49     window_sh = 30
50
51     def rolling_sharpe(ret_arr, w):
52         result = [np.nan] * (w - 1)
53         for i in range(w - 1, len(ret_arr)):
54             r = ret_arr[i - w + 1: i + 1]
55             s = (np.mean(r) - rf_d) / (np.std(r, ddof=1)
56                 + 1e-10)
57             result.append(s * math.sqrt(252))
58         return result
59
60     rs_mcts = rolling_sharpe(ret_mcts, window_sh)
61     rs_bah = rolling_sharpe(ret_bah, window_sh)
62     d_roll = self.dates[1:] # retornos tienen T-1
63                          puntos
64
65     ax3.plot(d_roll, rs_mcts, lw=1.4, color="#1f77b4",
66            label="MCTS")

```

```

60     ax3.plot(d_roll, rs_bah, lw=1.0, color="#ff7f0e",
61             ls="--", label="B&H")
62     ax3.axhline(0, color="gray", lw=0.8, ls=":")
63     ax3.fill_between(d_roll, rs_mcts, 0,
64                     where=np.array(rs_mcts) > 0,
65                     color="#1f77b4", alpha=0.15)
66     ax3.set_title(f"Sharpe rodante ({window_sh}d)")
67     ax3.set_ylabel("Sharpe anualizado")
68     ax3.set_xlabel("Fecha")
69     ax3.legend(fontsize=9)
70     ax3.grid(alpha=0.3)
71
72     plt.tight_layout()
73     path = os.path.join(self.output_dir, "fig3_returns.
74                         png")
75     plt.savefig(path, dpi=150, bbox_inches="tight")
76     plt.close()
77     print(f"[OK] {path}")

```

Listing 14: Figura 3 – retornos y rolling Sharpe

9.4. Figura 4 — Árbol MCTS y convergencia UCB

Fig. 4: fig4_tree.png

- **Panel izquierdo:** Árbol de decisión MCTS del último día evaluado (profundidad 3). Los nodos se colorean por acción y muestran n (visitas) y $\bar{w}$ (recompensa media).
- **Panel derecho superior:** Recompensa media acumulada por iteración ($\bar{w}_k = \frac{1}{k} \sum_{i=1}^k r_i$). Muestra si el árbol convergió.
- **Panel derecho inferior:** Distribución de acciones por mes (barras apiladas %).

```

1  def plot_mcts_tree(self) -> None:
2      """Fig 4: arbol de decision + convergencia UCB +
3          acciones por mes."""
4      root = self.results["ucb_root_last"]
5      iter_rewards = self.results["iter_rewards_last"]
6      actions = self.results["actions"]
7
8      fig = plt.figure(figsize=(16, 9))
9      fig.suptitle(f"Interior del arbol MCTS -- {self.
10                  ticker}",
11                  fontsize=13, fontweight="bold")
12      gs = gridspec.GridSpec(2, 2, hspace=0.45, wspace
13                             =0.35)
14
15      # Panel 1 (columna izquierda, toda la altura): arbol
16      ax1 = fig.add_subplot(gs[:, 0])
17      ax1.axis("off")

```



```

15     ax1.set_title("Arbol de decisiones (profundidad 3)",
16                   fontsize=10)
17     self._draw_tree(ax1, root, max_depth=3)
18
19     # Panel 2 (derecha arriba): convergencia
20     ax2 = fig.add_subplot(gs[0, 1])
21     cumul = np.cumsum(iter_rewards) / (np.arange(len(
22         iter_rewards)) + 1)
23     ax2.plot(range(1, len(cumul) + 1), cumul,
24             color="#8e44ad", lw=1.5)
25     ax2.set_title("Convergencia UCB (recompensa media
26         acumulada)")
27     ax2.set_xlabel("Iteracion MCTS")
28     ax2.set_ylabel(r"$\bar{w}_k$")
29     ax2.grid(alpha=0.3)
30
31     # Panel 3 (derecha abajo): acciones por mes
32     ax3 = fig.add_subplot(gs[1, 1])
33     dates_act = pd.DatetimeIndex(self.dates[:len(actions)
34         ])
35     df_act = pd.DataFrame(
36         {"month": dates_act.to_period("M"),
37          "action": [a.name for a in actions]}
38     )
39     counts = (df_act.groupby(["month", "action"])
40         .size().unstack(fill_value=0))
41     counts_pct = counts.div(counts.sum(axis=1), axis=0) *
42         100
43
44     colors_act = {
45         "BUY_25": "#1a7a1a", "BUY_10": "#2ecc71",
46         "HOLD": "#3498db",
47         "SELL_10": "#e74c3c", "SELL_25": "#8e0000",
48     }
49     bottom = np.zeros(len(counts_pct))
50     for col in ["BUY_25", "BUY_10", "HOLD", "SELL_10", "
51         SELL_25"]:
52         if col in counts_pct.columns:
53             vals = counts_pct[col].values
54             ax3.bar(range(len(counts_pct)), vals, bottom=
55                 bottom,
56                     color=colors_act[col], label=col,
57                     alpha=0.85)
58             bottom += vals
59     ax3.set_title("Distribucion de acciones por mes (%)")
60     ax3.set_ylabel("%")
61     ax3.set_xticks(range(len(counts_pct)))
62     ax3.set_xticklabels(
63         [str(p) for p in counts_pct.index],
64         rotation=45, ha="right", fontsize=7,
65     )

```

```

58     ax3.legend(fontsize=7, loc="upper right")
59
60     plt.tight_layout()
61     path = os.path.join(self.output_dir, "fig4_tree.png")
62     plt.savefig(path, dpi=150, bbox_inches="tight")
63     plt.close()
64     print(f"[OK] {path}")
65
66     def _draw_tree(self, ax, node: Node, max_depth: int = 3)
67     -> None:
68         """Dibuja el arbol MCTS recursivamente en los ejes ax
69         ."""
70
71         ACTION_COLORS = {
72             Action.BUY_25: "#1a7a1a", Action.BUY_10: "#2
73             ecc71",
74             Action.HOLD: "#3498db",
75             Action.SELL_10: "#e74c3c", Action.SELL_25: "#8
76             e0000",
77             None: "#95a5a6",
78         }
79
80     def _draw(node, x, y, dx, depth):
81         if depth > max_depth:
82             return
83         mean_r = node.w / node.n if node.n > 0 else 0.0
84         label = (
85             f"{node.action.name if node.action else 'ROOT'
86             }\n"
87             f"n={node.n} w={mean_r:.3f}"
88         )
89         color = ACTION_COLORS.get(node.action, "#95a5a6")
90         ax.text(x, y, label, ha="center", va="center",
91             fontsize=7,
92             bbox=dict(boxstyle="round,pad=0.35",
93                 facecolor=color, alpha=0.85,
94                 edgecolor="gray"))
95
96         n_ch = len(node.children)
97         if n_ch == 0:
98             return
99         child_dx = dx / max(n_ch, 1)
100         x_start = x - dx / 2 + child_dx / 2
101         for i, child in enumerate(node.children):
102             cx, cy = x_start + i * child_dx, y - 1.4
103             ax.annotate("", xy=(cx, cy + 0.3), xytext=(x,
104                 y - 0.3),
105                 arrowprops=dict(arrowstyle="->",
106                     color="gray", lw
107                     =0.9))
108             _draw(child, cx, cy, child_dx, depth + 1)
109
110         _draw(node, x=0.5, y=max_depth * 1.4, dx=1.0, depth
111             =0)

```

Listing 15: Figura 4 – arbol MCTS y analisis de convergencia

9.5. Figura 5 — Trayectorias GBM del rollout

Fig. 5: fig5_gbm_paths.png

- **Panel único:** 50 trayectorias GBM simuladas desde el precio actual de TSLA al horizonte `rollout_days`. Se muestran: media de la simulación, banda $\pm 1\sigma$, percentiles 5% y 95%, y la línea de precio actual.

Qué muestra: la distribución de precios futuros que el agente usa internamente en el rollout del último día evaluado.

```

1  def plot_gbm_paths(self) -> None:
2      """Fig 5: trayectorias GBM del rollout (ultimo dia).
3          """
4      paths = self.results["gbm_paths_last"] # shape
5              (N_PATHS, rollout_days)
6      S0 = float(self.prices[-1])
7      n_days_sim = paths.shape[1]
8      t_axis = np.arange(n_days_sim + 1)
9      # Prefijo con el precio actual
10     full = np.hstack([np.full((paths.shape[0], 1), S0),
11                        paths])
12
13     mean_path = np.mean(full, axis=0)
14     std_path = np.std(full, axis=0)
15     p05 = np.percentile(full, 5, axis=0)
16     p95 = np.percentile(full, 95, axis=0)
17
18     fig, ax = plt.subplots(figsize=(12, 6))
19     fig.suptitle(
20         f"Trayectorias GBM del rollout -- {self.ticker}\n"
21         f"(distribucion de precios futuros usada"
22         f"internamente por MCTS)",
23         fontsize=12, fontweight="bold",
24     )
25
26     for path in full:
27         ax.plot(t_axis, path, color="#1f77b4", lw=0.4,
28                 alpha=0.25)
29
30     ax.plot(t_axis, mean_path, color="#d62728", lw=2.0,
31             label="Media GBM")
32     ax.fill_between(t_axis, mean_path - std_path,
33                     mean_path + std_path,
34                     color="#d62728", alpha=0.15, label=r"$\pm 1\sigma$")

```

```

28     ax.plot(t_axis, p05, color="#2ca02c", lw=1.2,
29             ls="--", label="Percentil 5%")
30     ax.plot(t_axis, p95, color="#2ca02c", lw=1.2,
31             ls="--", label="Percentil 95%")
32     ax.axhline(S0, color="black", lw=1.0, ls=":",
33               label=f"Precio actual ({S0:.2f} $)")
34
35     ax.set_xlabel("Dias de simulacion")
36     ax.set_ylabel(f"Precio simulado {self.ticker} [$]")
37     ax.legend(fontsize=9)
38     ax.grid(alpha=0.3)
39
40     plt.tight_layout()
41     path_out = os.path.join(self.output_dir, "
42                             fig5_gbm_paths.png")
43     plt.savefig(path_out, dpi=150, bbox_inches="tight")
44     plt.close()
45     print(f"[OK] {path_out}")

```

Listing 16: Figura 5 – visualizacion de trayectorias GBM

10. Sección 8 — Tabla de métricas en consola

```

1  def print_metrics_table(m_mcts: dict, m_bah: dict) -> None:
2      """Imprime la comparativa de metricas en formato tabular.
3      """
4      sep = "=" * 58
5      print(sep)
6      print("EVALUACION DE POSICION -- MCTS vs BUY & HOLD")
7      print(f"Activo: {TICKER} Periodo: {PERIOD}")
8      print(sep)
9      print(f"{'Metrica':<22} {'MCTS Agent':>14} {'Buy & Hold'>14}")
10     print("-" * 58)
11
12     def p(label, key, fmt="+.2%"):
13         v_m = m_mcts[key]
14         v_b = m_bah[key]
15         print(f"{'Metrica':<22} {format(v_m, fmt):>14} {format(v_b, fmt):>14}")
16
17     p("Valor final [$]", "final", ".2f")
18     p("ROI acumulado", "roi")
19     p("CAGR", "cagr")
20     p("Sharpe (anual)", "sharpe", ".4f")
21     p("Sortino (anual)", "sortino", ".4f")
22     p("Max Drawdown", "max_dd")
23     p("Calmar", "calmar", ".4f")

```

```

23 p("VaR 95%% (diario)", "var_95")
24 p("CVaR 95%% (diario)", "cvar_95")
25 print(sep)

```

Listing 17: Funcion print_metrics_table()

La salida en consola tiene el siguiente aspecto:

```

=====
EVALUACION DE POSICION -- MCTS vs BUY & HOLD
Activo: TSLA    Periodo: 2y
=====

```

Metrica	MCTS Agent	Buy & Hold
Valor final [\$]	12,450.00	11,200.00
ROI acumulado	+24.50%	+12.00%
CAGR	+11.82%	+5.83%
Sharpe (anual)	1.4321	0.9812
Sortino (anual)	2.1034	1.2300
Max Drawdown	-18.34%	-32.10%
Calmar	0.6445	0.1813
VaR 95% (diario)	-2.34%	-3.10%
CVaR 95% (diario)	-3.89%	-5.20%

```

=====

```

11. Sección 9 y 10 — Reporte y bloque principal

```

1 def save_report(m_mcts: dict, m_bah: dict,
2                 results: dict, output_dir: str) -> None:
3     """Genera backtest_report.md con metricas y referencias a
4     figuras."""
5     path = os.path.join(output_dir, "backtest_report.md")
6     lines = [
7         "# Backtest MCTS -- Evaluacion de posicion en TSLA",
8         f"> Generado el {datetime.date.today().isoformat()}",
9         "",
10        "## Configuracion",
11        f"| Parametro | Valor |",
12        f"|---|---|",
13        f"| Ticker | '{TICKER}' |",
14        f"| Periodo | {PERIOD} |",
15        f"| Iteraciones MCTS | {results.get('mcts_iterations', '--')} |",
16        f"| Horizonte rollout | {results.get('rollout_days', '--')} dias |",
17        f"| Capital inicial | {results['initial_cash']:.2f} $ |",
18        ""

```

```

18     "## Resultados",
19     "| Metrica | MCTS | Buy & Hold |",
20     "|---|---|---|",
21     f"| ROI | {m_mcts['roi']:+.2%} | {m_bah['roi']:+.2%} |",
22     f"| Sharpe | {m_mcts['sharpe']:.4f} | {m_bah['sharpe']:.4f} |",
23     f"| Max DD | {m_mcts['max_dd']:+.2%} | {m_bah['max_dd']:+.2%} |",
24     f"| VaR 95% | {m_mcts['var_95']:+.2%} | {m_bah['var_95']:+.2%} |",
25     "",
26     "## Figuras",
27     "![Senales](fig1_signals.png)",
28     "![Performance](fig2_performance.png)",
29     "![Retornos](fig3_returns.png)",
30     "![Arbol MCTS](fig4_tree.png)",
31     "![GBM Paths](fig5_gbm_paths.png)",
32 ]
33 with open(path, "w", encoding="utf-8") as f:
34     f.write("\n".join(lines))
35 print(f"[OK] Reporte: {path}")
36
37
38 # -- Bloque principal
39 -----
40 if __name__ == "__main__":
41     os.makedirs(RESULTS_DIR, exist_ok=True)
42
43     # 1. Datos
44     prices, dates = download_data(TICKER, PERIOD)
45
46     # 2. Backtest
47     bt = Backtester(
48         mcts_iterations = 300,
49         rollout_days    = 20,
50         window          = 40,
51         tx_cost         = 0.001,
52         rng_seed        = 42,
53     )
54     results = bt.run(prices, dates, initial_cash=INITIAL_CASH)
55
56     # 3. Metricas
57     m_mcts = Backtester.compute_metrics(results["portfolio_series"], "MCTS")
58     m_bah  = Backtester.compute_metrics(results["bah_series"], "B&H")
59     print_metrics_table(m_mcts, m_bah)
60

```

```
61     # 4. Figuras
62     viz = Visualizer(results, prices, dates, TICKER,
63                     RESULTS_DIR)
64     viz.plot_signals()           # fig1_signals.png
65     viz.plot_performance()      # fig2_performance.png
66     viz.plot_returns()          # fig3_returns.png
67     viz.plot_mcts_tree()        # fig4_tree.png
68     viz.plot_gbm_paths()        # fig5_gbm_paths.png
69
70     # 5. Reporte
71     save_report(m_mcts, m_bah, results, RESULTS_DIR)
72
73     print(f"\n[OK] Todo guardado en: {RESULTS_DIR}/")
```

Listing 18: Funcion `save_report()` y bloque `__main__`

12. Resumen comparativo de mejoras

Aspecto	mcts_trading.py	mcts_tsla.py
Espacio de acciones	3 (BUY / SELL / HOLD)	5 (BUY 25 %, BUY 10 %, HOLD, SELL 10 %, SELL 25 %)
Coste de transacción	No modelado	Sí, proporcional al capital operado
Rollout	GBM con μ, σ globales	GBM con calibración rolling de window días
Lookback bias	Sí (precio histórico en expansión)	No (precio GBM simulado en expansión)
Recompensa	Valor absoluto de la cartera	Log-retorno (más estable numéricamente)
Métricas	ROI + Sharpe	+ CAGR, Sortino, Calmar, VaR 95 %, CVaR 95 %
Visualizaciones	1 figura, 3 paneles	5 figuras, 2-3 paneles cada una
Convergencia UCB	No registrada	Gráfico de $\bar{w}_k$ por iteración
Trayectorias GBM	No visualizadas	Figura 5 dedicada (50 paths + bandas)
Ratio de posición	No calculado	Serie temporal completa (position_ratio)

Flujo de datos entre secciones

```
download_data() → Backtester.run() → dict results → Visualizer +
compute_metrics() + save_report()
```